

PA2.1. Implementing an RTSP client

After completing this assignment, you should be:

- Able to read and implement a protocol specification;
- Familiar with out-of-band data transmission;
- Able to work with a combination of UDP and TCP sockets;
- Able to visualize issues related to timing in multimedia streaming.

All policies listed on the eClass page for Assignments Policies and Procedures apply to this assignment.

Special Note

Although this assignment's autograder tests the most common input issues and produces an initial score, this score may be overridden by a manual review by the course staff. This manual review will ensure your code works not only based on the simple cases listed in the resulting tests, but also for other considerations listed in the RFC and for additional tests. TAs will also review your code quality in terms of clarity and use of comments.

Introduction

In this assignment you will implement a streaming video client for stored video. A specific video file is requested by the client, and streamed from the server over the network to the client. The client will then play the video as it arrives.

The client and server communicate requests and responses between each other using a simplified version of the Real-Time Streaming Protocol (RTSP) and send the video data using a subset of the Real-time Transfer Protocol (RTP). We have provided you with an executable JAR file with the implementation of an RTSP server and the user interface of the client. Your task is to implement the relevant features of the client side portion of the RTSP and RTP protocols. You are not allowed to modify any other part of the application other than what is necessary to provide network communication functionality.

When implementing this assignment you should write small pieces of code and then test and verify their functionality before proceeding. You are strongly encouraged to implement the assignments functionality incrementally in the order it is presented in the description.

This assignment is implemented in two parts: in the first part, you will implement the client's network connectivity, playing frames as they arrive without consideration of timing. In the second part of the assignment, you will improve the functionality of the player by implementing buffering techniques that allow frames to be played at a consistent rate, in the correct order, and properly handling missing frames.

Provided Files

The following files are provided to you to use during the implementation of this assignment:

- a basic implementation of the RTSP server in Java. The source code is not provided, but it is not necessary for the implementation of your code, neither should you depend on it.
 - [RTSPServer.jar](#)
- the starting code for the client implementation.
 - [RTSPClient.zip](#)
- sample video files for transmission. Note that the server accepts a very specific video format.
 - [movie1.Mjpeg](#)
 - [movie2.Mjpeg](#)
 - [movie3.Mjpeg](#)

Running the server

In this assignment you are provided with the server executable. In order to test your client, you should first start the server on a particular port. To start an RTSP server on Windows or Mac OS, you may double-click the JAR file in the location where you downloaded it. The server will request the port number in a dialog box. You should provide a valid TCP port number (ideally between 1025 and 65535). The server will then accept RTSP connections on the provided port.

Alternatively, if you prefer to run your server in a terminal, without a GUI (in "Headless mode"), you can use the following command:

```
java -Djava.awt.headless=true -jar RTSPServer.jar server_port
```

Where server_port is the port your server listens to for incoming RSP connections. Using 0 as the server_port will choose a random available port, which will be printed on the terminal.

The server will print some relevant information on its GUI Window (if running with a GUI), or on the terminal (if running in Headless mode).

For simplicity, make sure the video files you will use for transmission are saved in the same folder as the one you are running the server from. You may use any of the Mjpeg files provided above.

By default, the server will behave as expected, with a proper stream of frames, as described above. Since connections on the Internet aren't always well behaved, the server has options that simulates some of the problems a video streaming application might encounter. These are listed as "Funky Servers" in the server interface. You do not need to handle these servers in this part of the assignment, these servers will be useful in the second part.

Running the client

The ZIP file above contains a folder that can be imported into modern IDEs as a project. You may then run the application from inside the IDE, by ensuring that `src` is set as the source folder, and running the `ca.yorku.rtsp.client.ui.MainWindow` class.

Upon starting the client you will be presented with a dialog box to provide the hostname of the machine the server is running on (you can use `localhost` if the server is running on the same computer as the client), and the port number the server is listening on. The client will then establish a TCP connection to the server. This connection will be used for RTSP, but at this time no messages are sent or received.

Once connected to the RTSP server, the user interface will provide five buttons. Each button is associated with an RTSP operation, as listed in the `RTSPConnection.java` file. Details about what is expected to happen in each method are provided in the method comments, but a brief description is provided below:

- *Open*: asks for a file name and sends a SETUP request to the server. This command sets up the session and establishes an RTP datagram socket (using UDP).
- *Play*: sends a PLAY request to the server. This command starts the playback of the video in its first run, or continues a playback that was previously paused.
- *Pause*: sends a PAUSE request to the server. This command pauses the playback of the video.
- *Close*: sends a TEARDOWN request to the server. This command stops the video and closes the RTSP session and RTP connection, but maintains the RTSP connection open.
- *Disconnect*: closes the RTSP connection.

Note that you will maintain two different sockets open simultaneously, one regular (stream) socket for the RTSP connection and one datagram socket for the RTP connection. It is your responsibility to open, maintain and close these connections appropriately, and handle connection interruptions.

Note that, by design, these buttons are enabled in all states. This is done so you can properly test your code with unconventional orders of execution, like attempting to play a video twice, or pausing a video that is not currently playing. In most of these scenarios you should follow the described behaviour in the original method description.

Since this assignment uses only a subset of the functionality of RTSP and RTP, you don't need to understand these protocols in depth to complete this assignment. Besides other limitations, the video streaming is implemented using individual JPEG images to represent an individual video frame. There is no inter-frame compression, audio track, or control track. The server will, once the video ends, send an additional empty frame (with a header but with no payload) and then stop sending frames. The server will not send any message over the RTSP connection informing the client that the video ended. You are, naturally, not required to do any additional processing at the end of the video, other than stopping the thread that listens for frames.

The RTSP protocol

You are welcome to read the RTSP RFC (RFC 2326) if you wish to, but since this assignment only uses a small subset of this protocol, a simple example should be enough to understand the functionality required for this assignment. In the example below, the client's requests with a light yellow background, while the server's replies have a light grey background.

SETUP movie.Mjpeg RTSP/1.0

CSeq: 1

Transport: RTP/UDP; client_port= 25000

RTSP/1.0 200 OK

CSeq: 1

Session: 123456

PLAY movie.Mjpeg RTSP/1.0

CSeq: 2

Session: 123456

RTSP/1.0 200 OK

Seq: 2

Session: 123456

PAUSE movie.Mjpeg RTSP/1.0

CSeq: 3

Session: 123456

RTSP/1.0 200 OK

CSeq: 3

Session: 123456

PLAY movie.Mjpeg RTSP/1.0

CSeq: 4

Session: 123456

RTSP/1.0 200 OK

CSeq: 4

Session: 123456

TEARDOWN movie.Mjpeg RTSP/1.0

CSeq: 5

Session: 123456

RTSP/1.0 200 OK

CSeq: 5

Session: 123456

Note that the RTSP message format is somewhat similar to the HTTP format. As in HTTP, each request and each response ends with an empty line. The empty line indicates the end of the message. Although the RTSP protocol requires lines to be terminated by a carriage-return line-feed sequence (`\r\n`), the server also accepts a line-feed on its own (`\n`) as valid, and you are allowed to use either option.

The Session header in the PLAY, PAUSE and TEARDOWN must be the same as the one returned by the server in the response to the SETUP command. The value of the CSeq header is a number which is incremented by one for each request you send, and will have its value matched in the corresponding response.

The Transport header in the SETUP command is of particular relevance. The provided server will only accept the RTP/UDP transport method. Before sending the SETUP command, the client must open a new UDP (datagram) socket listening on a random port. The transport header will then include the listening

port number used by this socket. Note that you do not need to generate the random number yourself, the DatagramSocket class in Java has specific methods to listen to a random available port and to retrieve the port in use.

Once the PLAY command is issued, the datagram socket will receive video frames, one per packet, until a PAUSE or TEARDOWN command is issued or the video ends. Note that, based on the nature of UDP, it is possible that an incorrect port number might not trigger an error, but it might cause the frames to be lost.

The RTP protocol

The video streaming frames will be transferred using RTP packets. This assignment will use only a small subset of the RTP protocol. You are responsible for parsing the packet data and generate a Frame object based on that data. Each datagram packet will contain exactly one frame, corresponding to a JPEG image. The end of a video stream is indicated by a packet that contains the RTP header, but no payload.

When the server receives the PLAY request from the client, it starts a timer. Every 40ms (i.e., 25 times per second) the server will read one video frame from the file and send it to the client, encapsulated as an RTP packet. Your job is to read this packet and convert the information in that packet into appropriate values.

The format of the RTP packet header is described in section 5.1 of the [RFC for the RTP protocol](#), as well as in section 9.4.1 of the textbook. **You don't need to read the other sections of the RFC**, since they are not relevant to this assignment. In particular, note that padding (P), extension (X), CSRC count (CC) and synchronization source (SSRC) are not used in this assignment, and you can ignore their values. Also note that there are no CSRC headers, so the header ends after the SSRC identifier (i.e., the header contains exactly 12 bytes).

After the RTP packet header (i.e., starting at position 12 in the packet array), the payload (content of the frame) starts. The number of bytes in the payload is given by the total size of the datagram packet minus the header size. Note that you must use *the length of the datagram packet*, not the length of the underlying byte array, to determine the size of the image.

The numbers represented in the RTP packet are in network byte order (also known as big-endian). You must make sure that when you fill the integer fields of the frame (such as timestamp and sequence number), that the bytes are added in proper order. The ByteBuffer class may be useful to retrieve this data.

PA2.2. Improving the experience of a multimedia stream

This part of the assignment assumes you have completed the first part. Note that the implementation of the behaviour described below depends on a correct implementation of the RTSP connection requirements, so you will only be able to test this assignment after completing the first part. However, the autograder in this part will test your code against the solution of the first part (not your own implementation).

All policies listed on the eClass page for Assignments Policies and Procedures apply to this assignment.

Special Note

Although this assignment's autograder tests the most common input issues and produces an initial score, this score may be overridden by a manual review by the course staff. This manual review will ensure your code works not only based on the simple cases listed in the resulting tests, but also for other considerations listed in the RFC and for additional tests. TAs will also review your code quality in terms of clarity and use of comments.

The Funky Servers

In the first part of this assignment, you have only dealt with a well behaved video stream. Since connections on the Internet aren't always well behaved, we've created a server that simulates some of the problems a video streaming application might encounter.

To run the server with simulated problems, open the server as before, and then select one of the scenario buttons on the top. The REGULAR scenario (default) runs the server with no problems. The other scenarios (FUNKY_A, FUNKY_B, ... FUNKY_H) introduce predefined network problems, some mild, some intense.

If you are running the server in a command-line interface, just add a letter (A through H) as an additional argument, like the following:

```
java -Djava.awt.headless=true -jar RTSPServer.jar server_port A
```

After you complete the implementation of the first part of the assignment, run your client and interact with each of the "funky" scenarios. You should be able to observe some interesting behaviour based on how the simulated server or network treats the data, and notice how it affects the video playback experience. Examples of simulated events include differences in transmission rates, lost packets, delayed packets and burst of congestion.

In this part of the assignment, you will modify the behaviour of your client to mitigate the playback problems presented by the various funky server scenarios. Your client must try to deliver the best user experience possible under the circumstances. In other words, it must play the video as smoothly as possible.

There are several possible implementations that could improve the playback experience. Your implementation, however, will be based on modifying the Session class so that, instead of playing the

frames as they are received, it will instead buffer these frames and play them at an appropriate rate in the UI. All changes must be made in the Session class only.

Here is how your Session class must behave:

- When a new file is open (the SETUP command), the session will immediately retrieve frames from the server (the PLAY command), but it will not send these frames directly to the UI (represented by a set of listeners in the session).
- Every time the buffer reaches 100 frames, the session will request that the server stops sending new frames (the PAUSE command).
- When the user triggers the Play button in the user interface, the session will send to the UI (i.e., the listeners) individual frames at a constant rate of 25 frames per second (i.e., one frame every 40 milliseconds), removing each frame from the buffer as it is played.
- As frames are removed from the buffer, if the buffer size goes below 80 frames, the session must again request that the server resumes sending more frames (PLAY command).
- The session must serve individual frames in the order determined by the frame's sequence number. If frames are received by the session out of order, they must be reordered by the session's buffer. If an individual sequence number is missing, the UI should not be served a frame for that particular frame time (e.g., if you receive frames 1, 2, 4, 3, and 6, you must serve frame 1, then 40ms later frame 2, then 40ms later frame 3, then 40ms later frame 4, then skip frame 5 and 80ms later serve frame 6).
- If a frame is received with a sequence number lower than those that have already been played (it comes "too late"), it must be dropped.
- If the buffer becomes empty, the session must stop serving frames to the UI until the buffer has at least 50 frames to play or the server sends a message that the video ended; once that happens playback can resume.
- When the user triggers the Pause button in the user interface, the session will stop sending frames to the UI, but this will not directly affect the frame retrieval process.
- Closing the file or shutting down the connection must cause the session to stop playing frames. Opening a new file after that point must ensure the buffer is empty before new frames are retrieved.

Note that the behaviour above will mean that the state of the local UI/session will often be different than the state of the client connection, and that is ok. In essence, here is a summary of the possible states of the local session:

Last user action	Buffered frames	Connection state (client)	Send frames to UI
SETUP or PAUSE	less than 80	PLAY	No
	between 80 and 99	retain current state	No
	at least 100	PAUSE	No
PLAY	empty	PLAY	No
	between 1 and 49	PLAY	if already sending: continue; if play is called: start; if otherwise buffering: don't start.
	between 50 and 79	PLAY	Yes
	between 80 and 99	retain current state	Yes
	at least 100	PAUSE	Yes
PLAY (after end of stream)	empty	retain current state	No
	at least 1	retain current state	Yes
CLOSE or DISCONNECT	—	CLOSE or DISCONNECT	No

You are allowed to use any Java classes from the Java standard library (up to Java 17) to implement the features above. A suggested approach for the frame buffer is the use of a [SortedSet](#) (e.g., TreeSet) or [SortedMap](#) (e.g., TreeMap). The transmission of frames at a constant rate can be done with a [Timer](#) or a [ScheduledExecutorService](#).

You are strongly encouraged to make all your methods synchronized to avoid timing issues. While synchronization is outside the bounds of the learning goals of this course, in essence this keyword ensures that only one of these methods can be executing in a session at a particular time.